File Handling in C++

1. Files

- A **file** is collection of related **records**. A record contains data about an entity. A file can be used to save any type of data. Files are stored on secondary storage *(ROM)*. The data is stored in files permanently.
- Normally, a program inputs data from user and stores it in variables. The data stored in the variables is temporary. When the program ends, all data stored in variables is also destroyed. The user has to input data again from keyboard when the program is executed next time. The data has to be entered each time program is executed that wastes time. The user may also type wrong data at different times.
- A data file can be used to provide input to a program. It can also be used to store the output of a program permanently. If a program gets input from a file instead of keyboard, it will get the same data each time it is executed. There will be far less chance of errors.

1.1. Advantages of Files

Some important advantages of files are as follows:

- Files can store large amount of data permanently.
- Files can be updated easily.
- One file can be used by many programs for same input.
- Files save time and effort to input data via keyboard.

1.2. Types of Files

C++ provides two types of files. This categorization is based on how data is stored in files. Following are two types of files:

1. **Text Files**

   A type of file that stores data as readable and printable characters is called **text file**. A source program in C++ is an example of text file. The user can easily view and read the contents of a text file. It can also be printed to get a hard copy.

2. **Binary Files**

   A type of file that stores data as non-readable binary code is called **binary file**. An object file(in machine code) of a C++ is an example of binary file. The user cannot read the contents of a binary file. It can only be read and processed by computer.

2. File Access Methods

The way in which a file can be accessed is called **file access method**. It depends on the manner in which data is stored in files. Different file access method are as follows:

1. **Sequential Access Method**

Sequential access method is used to access data in the same sequence in which it is stored in a file. This method reads and writes data in a sequence. The length of each record is not fixed. It means that each record occupies different amount of memory. The *advantage* of this method is that the memory is not wasted. The *disadvantage* is that it takes more time to search a record. For example if the user needs to find the third record, he has to read the first two records also. The audio tapes are accessed sequentially.

2. File Access Methods (continue…)

2. Random Access Method

Random access method is used to access data item directly without accessing the preceding data. This method does not read and write data in sequence. It is very fast access method as compared to sequential access method. A searching operation with random access method takes less time to find the data item.

Random access files store data in fixed length records. The main *disadvantage* of this method is that fixed size of the record is used even if the user wants to store a small sized record. It wastes memory space. Data base management systems store files in a random access format. Similarly, audio CDs are accessed randomly.

3. Stream

A **stream** is a series of bytes associated with a file. It consists of data that is transferred from one location to another. Each stream is associated with a specific file by using the **open** operation. The information can be exchanged between a file and the program once a file is opened. The exchange is performed with the help of streams.

3.1 Types of Streams

The are two main types of streams i.e., **input stream** and **output stream**. The streams are automatically established when a program starts execution. The user can use them to input and output data at any time. Following are two main streams in C++ language:

1. **Standard Input Stream**

Standard input stream is used to input data. It establishes a connection between the standard input device and a program. The act of reading a data from an input device is called **extraction**. It is performed using **extraction operator >>** with **cin** object. The **cin** is an object of **istream** and handles input.

2. **Standard Output Stream**

Standard output stream is used to output data. It establishes a connection between the standard output device and a program. The act of writing data to an output stream is called **insertion**. It is performed using **insertion operator <<** with **cout** object. The **cout** is an object of **ostream** and handles output.

3.2 Predefined Stream Objects

C++ provides four predefined streams. These streams are opened when C++ program is executed. The predefined streams are as follows:

Stream Name	Used for	Linked to
cin	Standard input	Keyboard
cout	Standard output	Monitor
cerr	Standard error	Monitor
clog	Buffered error display	Monitor

3.3 Stream Class Hierarchy

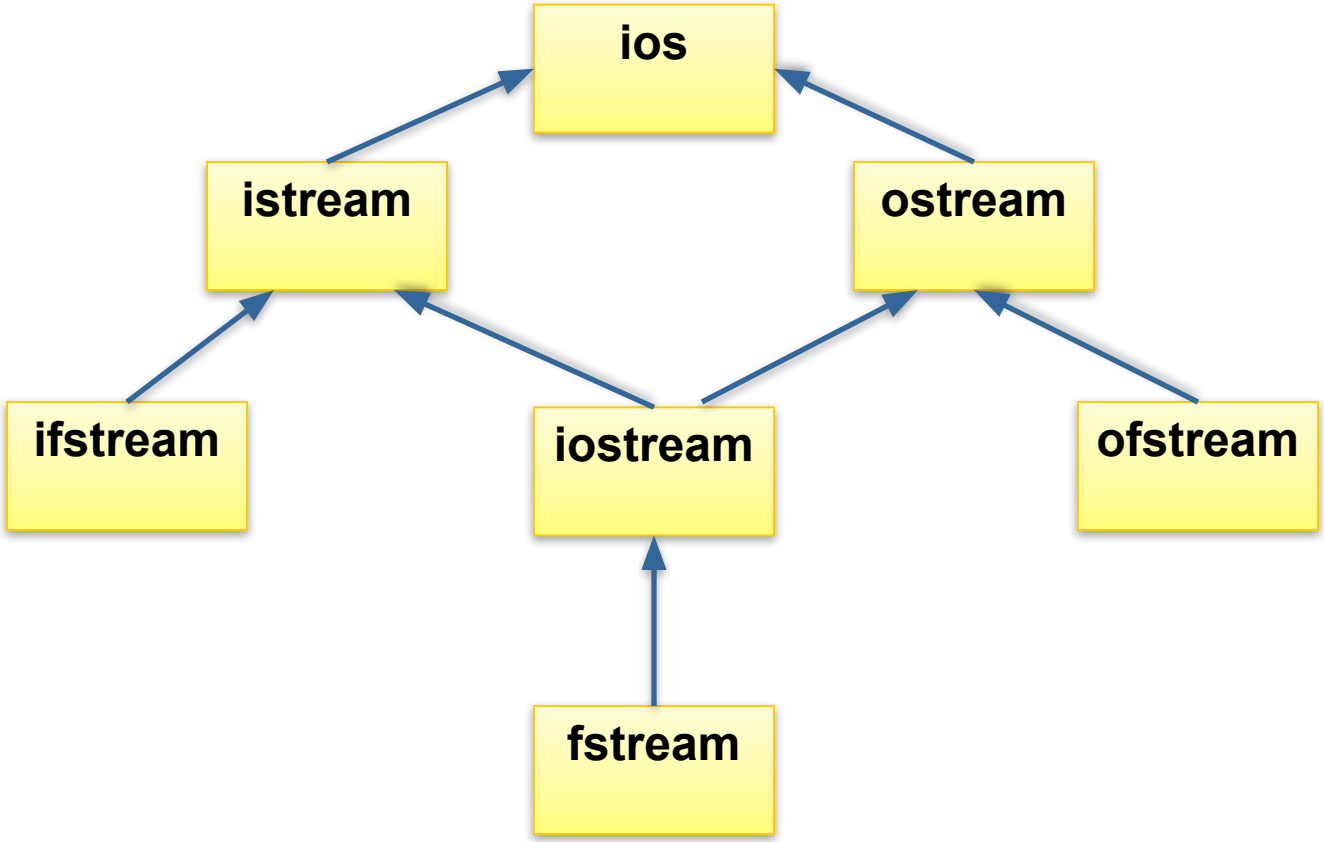
The stream classes are arranged in a hierarchy. C++ provides the following classes to perform output and input of characters with files:

- **ofstream:** This stream class is used to write on files.
- **ifstream:** This stream class is used to read from files.
- **fstream:** This stream class is used to both read and write from/to files.

These classes are derived directly or indirectly from the classes **istream** and **ostream**. The **cin** object used for input is an object of class **istream**. The **cout** object used for output is an object of class **ostream**.

3.3 Stream Class Hierarchy (continue…)

The stream class hierarchy is as follows:



4. Opening Files

- A file should be opened before it can be processed. A file can be opened by first creating an object of **ifstream**, **ofstream** or **fstream**. The object is then associated with a real file.
- An open file is represented by a stream object in a program. Any input or output operation performed on this stream object is applied to the physical file associated to it.

Syntax

The member function **open()** of stream object is used to open a file as follows:

open(filename, mode);

filename: it is name of the file to be opened.

mode: it is the mode in which the file is to be opened. It is optional parameter.

4. Opening Files (contd..)

Different types of modes for opening a file are as follows:

Mode	Description
ios::in	It is used to open a file for input operations.
ios::out	It is used to open a file for output operations.
ios::binary	It is used to open a file in binary mode.
ios::ate	It is used to set the initial position at the end of the file. If this flag is not set to any value, the initial position is the beginning of the file.
ios::app	It is used to perform all output operations at the end of the file, appending the content to the current content of file. It can only be used in streams opened for output-only operations.
ios::trunc	Any current content is discarded, assuming a length of zero on opening
ios::nocreate	It is used to open file only if it exists. The file is not created if it does not exist.

Table 1

4. Opening Files (contd..)

All of the flags shown in table 1 previous slide can be combined using pipe sign (|). For example, the following statements will open the file **test.txt** in binary mode to write data at the end of the file.

```
ofstream Test;
Test.open ("test.txt", ios::out | ios::app | ios::binary);
```

The three files **ifstream**, **ofstream** and **fstream** have constructors that automatically call the **open()** member function. The constructors have same parameters as open() function. The following format can also be used to open a file:

```
ofstream Test ("test.txt", ios::out | ios::app | ios::binary);
```

The above statement combines object construction and stream opening in a single statement. Both forms of opening a file are valid and equivalent.

4.1 Default Opening Modes

The member function open() in the classes of **ifstream**, **ofstream** and **fstream** uses a default mode to open a file if mode parameter is not used by the user. The default modes of these classes are given in table 2 below:

Class	Default mode parameter
ofstream	ios::out
ifstream	ios::in
fstream	ios::in | ios::out

Table 2

The default value is used only if the **open()** function is called without any values for the mode parameter.

5. Closing Files

The opened file should be closed when the input or output operations on the file are finished. The file should be closed so that its resources become available again. The member function close() of stream object is used to close a file. It takes no parameters. The function is used as follows:

```
Test.Close();
```

6.1 Writing Data to Files

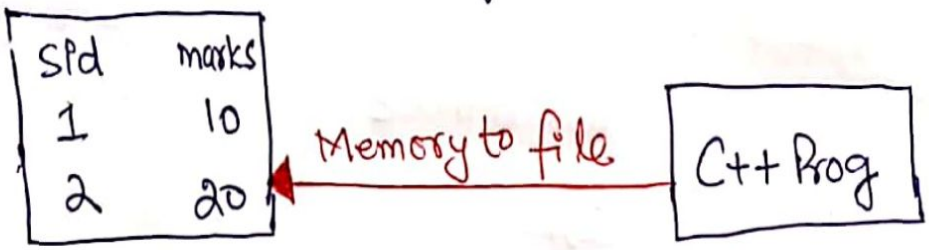
The **insertion operator <<** is used with **cout** object. It is frequently used in programs to write the output on the screen. The same operator can also be used to write the data in files. The operator is used with a stream object of **ostream** or **fsteam** to write data to files.

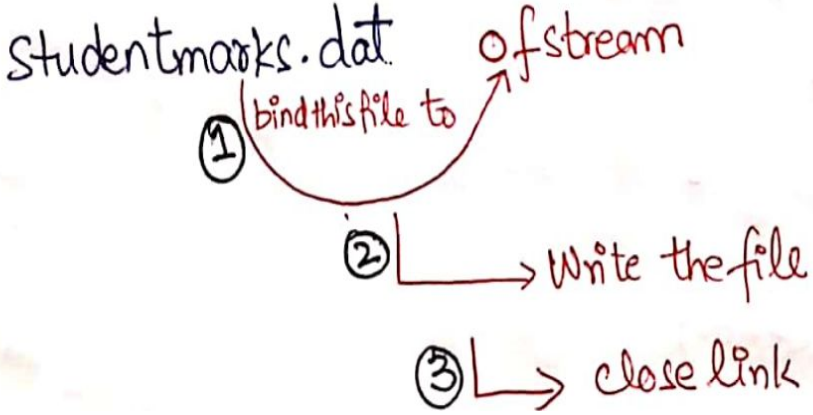
6.1.1 Example

Example to write Content to a file:

(1)

→ Suppose there is a file on my disk and the name of that file is Studentmarks.dat and we have stored some information related to student in this file.





6.1.2 Program 1

(2)

Program 1 - To Write Data to a file ofstream class.

```
#include <iostream>
#include <fstream>          → Include Header file
using name space std;
Pnt main ()
{
    ofstream fout;          → creating an object "fout" of ofstream
    fout.open ("studentmarks.dat", ios::out);
                                  ↳ file name        → output mode
    char ch='y';
    Pnt      sid  ,marks;
    while (ch=='y' || ch=='Y')  → As long as user enters character 'y' or 'Y', we will keep writing the records
                                  (i.e., until user enter 'y' or 'Y')
    {
        cout<<"Enter Student Reg no :"<<endl;   (registration)
        cin>> sid ;
        cout<<" Enter Student Marks :"<<endl;
        cin>>marks;
        fout<<sid<<endl<<marks<<endl;   → To write values of sid and marks into file.
        cout<<" Do you want to enter more records? (y/n):";
        cin>>ch;
    }
    fout.close();           → close the link
    return 0;
}
```

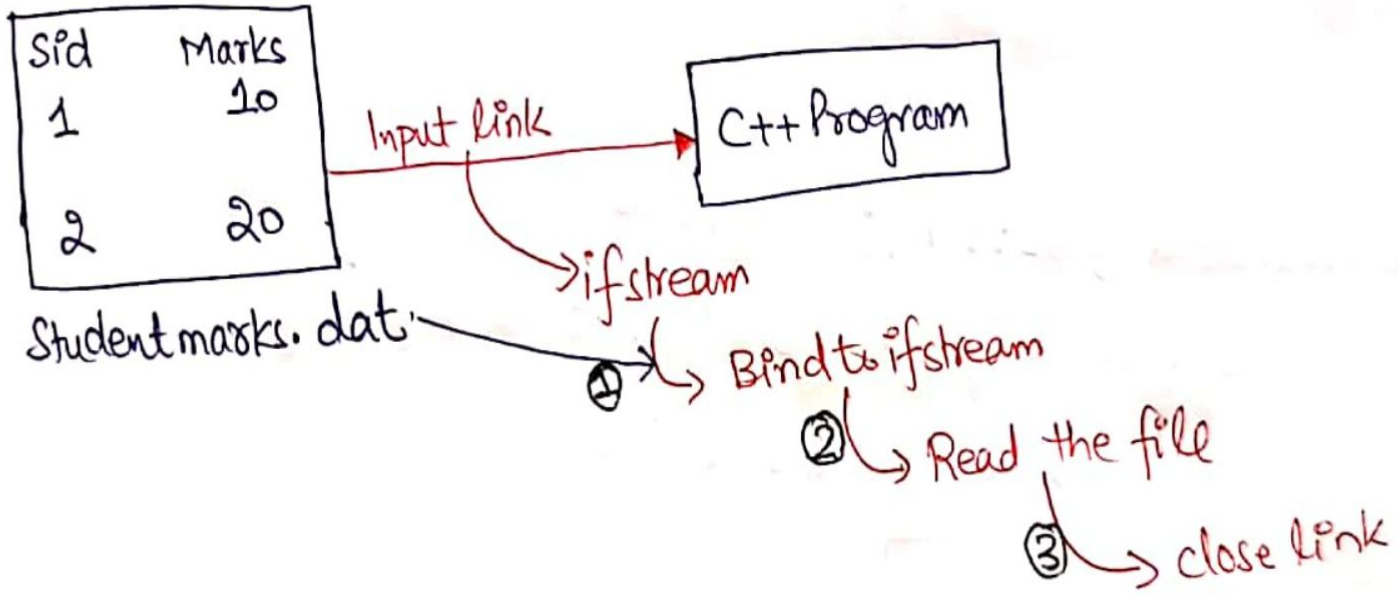
6.2 Reading Data From Files

The **extraction operator >>** is used with **cin** object. It is frequently used in programs to read input from the keyboard. The same operator can also be used to read data from files. The operator is used with a stream object of **istream** or **fsteam** to read data from files.

6.2.1 Example

Example to read from file:

→ Suppose we want to read the same file as in program 1:



6.2.2 Program 2

Program2 Write a program to read Data from File using ifstream Class.

```
#include<iostream>
#include<fstream>                    ]=include Header file.
using name space std;
int main ()
{
  ifstream fin;                      → creating an object "fin" of ifstream
  fin.open ("studentmarks.dat" ,ios::in);   → file name, → input mode
  fin.seekg(0); // This is used to bring the file pointer at the beginning
  int sid, marks;
  for (int i=0; i<2; i++)            → studentmarks.dat file
   {                                   for i=0 { 1 — sid
     fin>>sid;                                   10 — marks
     fin>>marks;                       for i=1 { 2 — sid
                                                 20 — marks
  cout<<"Student Id is:"<<sid;
  cout<<"\t Marks:"<<marks<<"\n";
  }
    fin.close();
    return 0;
  }
```

6.3 Example of Read and Write from/to Files

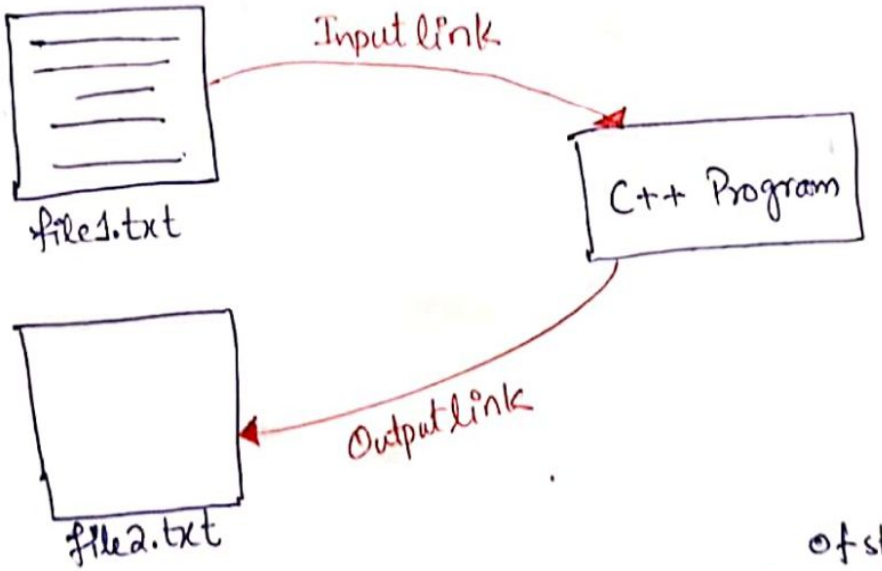
Example of Read & Write

Now we will see how to read from one file and write the contents into another file.

Suppose we have one file named "file1.txt", the requirement is read the contents of file1.txt and then write these contents (contents of file1.txt) into another file named "file2.txt".

for Input link

```
ifstream fin("file1.txt", ios::in);
```



for Output link

```
ofstream fout("file2.txt", ios::out);
```

```
while(!fin.eof())
{
    char c = fin.get();
    fout << c;
}
```

⇒ This loop will be executed until file1.txt is ended/finished.

∴ eof = End of file

(READ) Read character by character

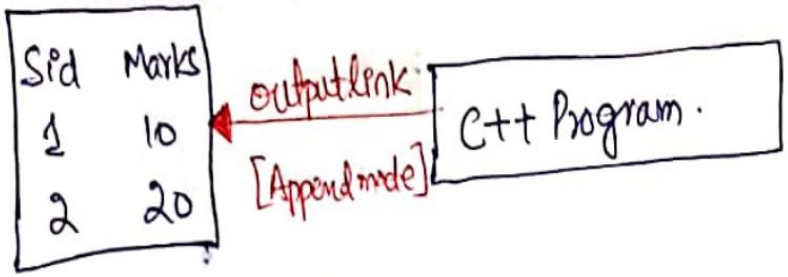
(WRITE) Write character by character

TRY IT YOURSELF & write C++ code for this example

6.4 Example to Append Data File

Example to Append the file

→ to append means retaining the old data.



Studentmarks.dat

Requirement: We have to enter records of some more students

→ I simply can't write records of new students into file because in this way the old records of data will be removed.

→ The link will be output

f1.txt → suppose we open it in output mode.

↳ i.e, whenever we write something into file 'f1.txt', the old data will be removed.

→ so if we want to retain the old data, then we need to append the file.

6.4.1 Program 3

Program3 Write a program to append data File

```
#include<iostream>
#include<fstream>          // include header file
using namespace std;
int main()
{
  ofstream fout;           // creating an object 'fout' of ofstream
fout.open("studentmarks.dat", ios::app);   // ios::app -> Append mode
                           // studentmarks.dat -> filename
   char ch='y';
   int sid, marks;
   while(ch=='y'||ch=='Y')   // This loop will be executed as long as user keeps entering 'y' or 'Y'.
    {                        // appending record i.e., user keeps entering 'y' or 'Y'.
      cout<<"Enter Student Id:\n";
      cin>>sid;
      cout<<"Enter Student Marks\n";
      cin>>marks;
     fout<<sid<<'\n'<<marks<<'\n';
     cout<<"\nDo you want to enter more records?(y/n):";
     cin>>ch;
    }
     fout.close();
     return 0;
}
```

References

1. *"C++ Programming: From Problem Analysis to Program Design",6/ed by D.S. Malik, (2013), CENGAGE Learning.*
2. *"Object Oriented Programming in C++," 4/ed by Robert Lafore (2001) .*
3. *"C++ How to Program," 5/ed by Deitel & Deitel (2005).*
4. *"Program with C++ Object Oriented Programming Aikman Series", by C.M Aslam and T.A Qureshi.*
5. *Related documents from open source, mainly internet.*